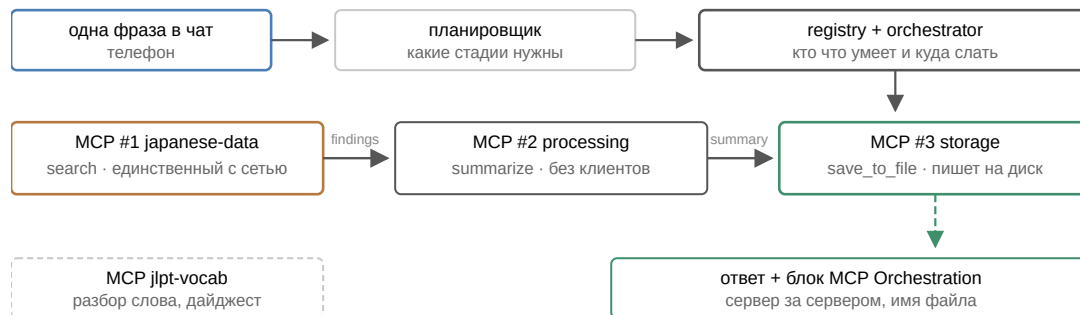


Задача

Разнести инструменты по **разным MCP-серверам** и научить агента работать с ними одновременно: узнавать, что у кого есть, выбирать нужный сервер и вести длинный флоу от запроса до файла — не путая порядок и не продолжая цепочку на испорченных данных.



четвёртый сервер остался с Дней 17-18: агент выбирает между ним и цепочкой

Решение

Сервер на стадии. Три процесса, которые ничего не знают друг о друге: *japanese-data* (единственный ходит в JLPT API), *processing* (внутри нет ни HTTP-клиента, ни доступа к диску — поэтому сводка не может разойтись с находками) и *storage* (единственный пишет). Общие модели данных — контракт между серверами — лежат в одном месте, поэтому дубли инфраструктуры нет.

Агент спрашивает, а не помнит. Список tools нигде не записан: каждый зарегистрированный сервер спрашивают через `list_tools()`, и таблица маршрутизации строится из ответов. Сервер, сменивший набор инструментов, сам меняет список у агента; сервер, который не поднялся, уносит с собой только свои tools — остальные продолжают работать.

```
async def call(self, tool, arguments):
    name = await self.server_of(tool)          # discovery решает, куда слать
    return await call_with_timeout(
        call_server_tool(tool, arguments, self.spec(name).parameters())
    )
```

Пять способов остановиться, и каждый называет себя в логге и в промпте: сервер недоступен; tool не найден ни на одном сервере; tool отработал и упал; сервер замолчал (таймаут); стадия вернула то, что следующей не годится. Последнее MCP поймать не может — вызов-то прошёл, испортились данные, — поэтому результат проверяют до передачи дальше.

Результаты

1 · ТРИ СЕРВЕРА, КАЖДЫЙ САМ ПО СЕБЕ

```
# .venv/bin/python3 -m app.services.mcp_client --server japanese-data
MCP: session initialized - server 'japanese-data' version 1.0.0, protocol 2025-11-25
MCP: list_tools() returned 1 tool(s) from 'japanese-data' - search(query)
# ... --server processing
MCP: session initialized - server 'processing' ... 1 tool(s) - summarize(findings)
# ... --server storage
MCP: session initialized - server 'storage' ... 1 tool(s) - save_to_file(summary, findings)
```

Три отдельных процесса, три разных имени сервера в рукопожатии, по одному инструменту в каждом.

2 · ТАБЛИЦА МАРШРУТИЗАЦИИ — РЕЗУЛЬТАТ DISCOVERY, А НЕ КОНФИГ

```
# python3 -c "... McpServerRegistry().routing_table() ..."
search                -> japanese-data
summarize              -> processing
save_to_file           -> storage
get_japanese_word_info -> jlpt-vocab
create_periodic_digest -> jlpt-vocab
get_latest_digest      -> jlpt-vocab
```

Шесть инструментов на четырёх серверах. Агент видит их одним списком, но каждый вызов уходит туда, где этот инструмент есть.

3 · ВТОРОЙ СЕРВЕР РАБОТАЕТ БЕЗ СЕТИ

```
# ... --server processing --call summarize --args '{"findings": {...}}'
MCP: summarize returned {"query": "学習", "headline": "'学習': 1 JLPT entry, N3 x1.", "summary": "'学習': 1 JLPT entry, N3 x1. 学習 - がくしゅう (gakushū) - study, learning - N3.", "based_on": 1, "levels": {"N3": 1}, "searched_at": "", "summarized_at": "2026-09-25T19:22:09+00:00"}
```

Findings поданы с руки — и сводка сделана из них. Ни одного HTTP-запроса в выводе, а пустой `searched_at` честно говорит, что поиска не было: этот сервер умеет только обрабатывать.

4 · ОДНА ФРАЗА В ПРИЛОЖЕНИИ — ВЕСЬ ДЛИННЫЙ ФЛОУ

Найди информацию о `学習`, сделай краткую сводку, сохрани результат и сообщи, что было сохранено.

MCP tool: `search(学習)`

MCP tool: `summarize({query=学習, found=true, count=1.0, matches=[{word=学習, reading=がくしゅう, romaji=gakushū, ...}], searched_at=2026-09-25T19:23:31+00:00})`

MCP tool: `save_to_file({query=学習, headline='学習': 1 JLPT entry, N3 x1., summary=..., summarized_at=2026-09-25T19:23:32+00:00}, {query=学習, found=true, ...})`

«Здравствуйте! На основе вашего запроса был успешно выполнен автоматический цикл обработки данных (поиск, суммаризация и сохранение). **1. Поиск (Search):** в базе данных JLPT найдено 1 совпадение, уровень N3. **2. Сводка (Summary):** ... **3. Сохранение (Save):** результаты поиска и сводки успешно сохранены в файл 20260925T192332-学習.json»

MCP ORCHESTRATION

- ✓ **Japanese Data Server**
search completed
- ✓ **Processing Server**
summarize completed
- ✓ **Storage Server**
save_to_file completed
- Pipeline completed**

Found: 学習 · がくしゅう (gakushū) · study, learning · N3

Summary: '学習': 1 JLPT entry, N3 x1. 学習 - がくしゅう (gakushū) - study, learning - N3.

Saved: 20260925T192332-学習.json

Имя сервера в каждой строке — слова бэкэнда: телефон не знает, какой сервер что предлагает, и подписывает ровно то, что ему прислали.

5 · ЧТО ДЕЛАЛ ОРКЕСТРАТОР

```
19:23:24 mcp_registry: [Orchestrator] Server japanese-data offers search
19:23:24 mcp_registry: [Orchestrator] Server processing offers summarize
19:23:25 mcp_registry: [Orchestrator] Server storage offers save_to_file
19:23:26 mcp_registry: [Orchestrator] Server jlpt-vocab offers get_japanese_word_info, create_periodic_digest, get_latest_digest
19:23:30 agent_pipeline: [Orchestrator] User request received: '学習'
19:23:30 agent_pipeline: [Orchestrator] Selected tool: search · Server: japanese-data · search completed
19:23:31 agent_pipeline: [Orchestrator] Selected tool: summarize · Server: processing · summarize completed
19:23:32 agent_pipeline: [Orchestrator] Selected tool: save_to_file · Server: storage · save_to_file completed
19:23:32 agent_pipeline: [Orchestrator] Pipeline completed: saved as 20260925T192332-学習.json
```

Discovery — один раз, при первом запросе; сама цепочка через три процесса уложилась в две секунды. В логе каждый шаг назван дважды: какой инструмент выбран и на какой сервер он отправлен.

6 · ДАННЫЕ ПЕРЕХОДЯТ МЕЖДУ СЕРВЕРАМИ ДОСЛОВНО

```
# journalctl -u gemini-image-api --since "10 min ago" | grep "call_tool summarize"
19:23:32 mcp_client: call_tool summarize({"findings": {"query": "学習", "found": true, "count": 1, "matches": [{"word": "学習", "reading": "がくしゅう", "romaji": "gakushū", "meaning": "study, learning", "jlpt_level": "N3"}]}, "source": "jlpt-vocab-api.vercel.app", "searched_at": "2026-09-25T19:23:31+00:00"})
# python3 -m json.tool "$(ls -t data/pipeline/*.json | head -1)"
{ "query": "\u5b66\u7fd2", "pipeline": ["search", "summarize", "save_to_file"],
  "summary": [{" ...", "searched_at": "2026-09-25T19:23:31+00:00", "summarized_at": "2026-09-25T19:23:32+00:00"
}],
  "findings": [{" "found": true, "count": 1, "matches": [{" "word": "\u5b66\u7fd2", "reading": "\u304c\u304d\u3057\u3085\u3046", "romaji": "gakush\u016b", ... }]} ] }
```

Аргумент второго вызова — дословно ответ первого, вплоть до отметки времени поиска; она же потом оказывается в файле. Японский в выводе *json.tool* показан escape-последовательностями — так печатает сама утилита, в файле записаны настоящие символы.

7 · И ЦЕПОЧКА ЗАПУСКАЕТСЯ НЕ ВСЕГДА

Объясни грамматику **～ながら**

«Грамматическая конструкция **～ながら** (nagara) — одна из важнейших в японском языке...» — **ни строк MCP, ни блока MCP Orchestration**: ни один сервер не понадобился.

ЧТО ЧЕМ ЗАКРЫТО

требование задания	чем подтверждено
несколько MCP-серверов	три отдельных процесса + четвёртый с Дней 17-18, разные имена в рукопожатии ✓
discovery tools каждого сервера	list_tools() по каждому, таблица маршрутизации собрана из ответов ✓
выбор нужного инструмента	на фразу про грамматику не вызван ни один ✓
маршрутизация запросов	search → japanese-data, summarize → processing, save_to_file → storage ✓
правильный порядок вызовов	1/3, 2/3, 3/3 в логе с отметками времени ✓
передача данных между серверами	аргумент каждого шага дословно равен ответу предыдущего ✓
длинный флоу до финального результата	от фразы до файла на диске — две секунды, ответ пользователю с именем файла ✓
Android ничего не оркестрирует	тот же /agent/chat; подписи серверов — слова бэкэнда ✓
ошибки и недоступность сервера	на видео не снималось — закрыто тестами (см. ниже)

Про последний пункт честно: падения в кадр не попали. Они закрыты тестами — упавший tool, сервер, который не стартует (discovery находит два других, цепочка останавливается на пропавшей стадии), сервер, исчезнувший между discovery и вызовом, и стадия с негодным результатом. По Дню 20 добавлено 16 тестов, всего в бэкенде 559 проходящих.

Вывод

Серверов несколько	три процесса, каждый со своим именем и одним инструментом, плюс прежний четвёртый
Маршрут не зашит	таблица построена из ответов серверов; агент знает только имя инструмента
Флоу идёт сам	одна фраза — три сервера, две секунды, файл на диске и его имя в ответе

День 19 связал инструменты в цепочку внутри одного сервера. День 20 разнёс их по разным процессам — и оказалось, что цепочке это не мешает: агент спрашивает, кто что умеет, отправляет каждый вызов по адресу и ведёт разговор до конца, а пользователь по-прежнему пишет одну фразу.